

# Klasifikacija cen mobilnih telefonov

Mila Kirova  
89241278

## Povzetek

V tem projektu sem razvila in primerjala modele strojnega učenja za napovedovanje cenovnega razreda mobilnih telefonov glede na njihove tehnične lastnosti. Uporabila sem Mobile Price Classification dataset, ki vsebuje 2000 primerov in 20 vhodnih značilk. Ciljna spremenljivka je price\_range s štirimi razredi: nizka, srednja, visoka in zelo visoka cena.

Najprej sem uporabila Decision Tree kot osnovni model, nato Random Forest kot naprednejši model in na koncu še umetno nevronske mreže MLP. Modele sem ocenila z natančnostjo, matriko zamenjav in classification reportom. Osnovni MLP model je dosegel natančnost 91,5 %, po dodatni izboljšavi z regularizacijo pa 97 %. Najboljši rezultat je dosegel izboljšani MLP model.

## 1. Uvod

Cilj projekta je bil razviti model strojnega učenja, ki napoveduje cenovni razred mobilnega telefona glede na njegove tehnične lastnosti. Problem spada med večrazredno klasifikacijo, saj ima ciljna spremenljivka price\_range štiri možne razrede: 0 pomeni nizko ceno, 1 srednjo ceno, 2 visoko ceno in 3 zelo visoko ceno.

V projektu sem najprej uporabila Decision Tree kot osnovni in lažje razumljiv model. Nato sem uporabila Random Forest, ki združuje več odločitvenih dreves in običajno zmanjša prenaučanje. Za dodatno primerjavo sem uporabila tudi umetno nevronske mreže MLP, da preverim, ali lahko doseže boljšo natančnost kot klasična modela strojnega učenja.

Glavni namen projekta je bil primerjati uspešnost teh modelov, analizirati njihove napake in ugotoviti, katere značilnosti najbolj vplivajo na napoved cenovnega razreda.

**Slika 1. Prikaz prvih petih primerov iz dataseta**

|   | battery_power | blue | clock_speed | dual_sim | fc | four_g | int_memory | m_dep | mobile_wt | n_cores | ... | px_height | px_width | ram  | sc_h | sc_w | talk_time | three_g | touch_screen | wifi | price_range |
|---|---------------|------|-------------|----------|----|--------|------------|-------|-----------|---------|-----|-----------|----------|------|------|------|-----------|---------|--------------|------|-------------|
| 0 | 842           | 0    | 2.2         | 0        | 1  | 0      | 7          | 0.6   | 188       | 2       | ... | 20        | 756      | 2549 | 9    | 7    | 19        | 0       | 0            | 1    | 1           |
| 1 | 1021          | 1    | 0.5         | 1        | 0  | 1      | 53         | 0.7   | 136       | 3       | ... | 905       | 1988     | 2631 | 17   | 3    | 7         | 1       | 1            | 0    | 2           |
| 2 | 563           | 1    | 0.5         | 1        | 2  | 1      | 41         | 0.9   | 145       | 5       | ... | 1263      | 1716     | 2603 | 11   | 2    | 9         | 1       | 1            | 0    | 2           |
| 3 | 615           | 1    | 2.5         | 0        | 0  | 0      | 10         | 0.8   | 131       | 6       | ... | 1216      | 1786     | 2769 | 16   | 8    | 11        | 1       | 0            | 0    | 2           |
| 4 | 1821          | 1    | 1.2         | 0        | 13 | 1      | 44         | 0.6   | 141       | 2       | ... | 1208      | 1212     | 1411 | 8    | 2    | 15        | 1       | 1            | 0    | 1           |

## 2. Materiali in metode

### 2.1 Podatki

Za projekt sem uporabila dataset Mobile Price Classification, ki je dostopen na platformi Kaggle. Dataset vsebuje 2000 primerov mobilnih telefonov in 21 stolpcev. Dvajset stolpcev predstavlja vhodne značilnosti, ciljna spremenljivka pa je price\_range.

Med vhodnimi značilnostmi so na primer zmogljivost baterije (battery\_power), hitrost procesorja (clock\_speed), notranji pomnilnik (int\_memory), število procesorskih jeder (n\_cores), ločljivost zaslona (px\_height in px\_width), količina RAM-a (ram) ter druge tehnične lastnosti mobilnih telefonov.

Ciljna spremenljivka price\_range vsebuje štiri razrede: 0 (nizka cena), 1 (srednja cena), 2 (visoka cena), 3 (zelo visoka cena).

Pregled podatkov je pokazal, da dataset ne vsebuje manjkajočih vrednosti. Vseh 2000 primerov je popolnih, zato dodatno čiščenje podatkov ni bilo potrebno.

```
data.shape #dobim stevilo vrstice in stolpcev
data.columns #dobim imena vseh stolpcev
data.info() #osnovne informacije o podatkih
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 21 columns):
#   Column              Non-Null Count  Dtype
---  -
0   battery_power        2000 non-null   int64
1   blue                 2000 non-null   int64
2   clock_speed          2000 non-null   float64
3   dual_sim             2000 non-null   int64
4   fc                   2000 non-null   int64
5   four_g              2000 non-null   int64
6   int_memory           2000 non-null   int64
7   m_dep                2000 non-null   float64
8   mobile_wt            2000 non-null   int64
9   n_cores              2000 non-null   int64
10  pc                   2000 non-null   int64
11  px_height            2000 non-null   int64
12  px_width             2000 non-null   int64
13  ram                  2000 non-null   int64
14  sc_h                 2000 non-null   int64
15  sc_w                 2000 non-null   int64
16  talk_time            2000 non-null   int64
17  three_g              2000 non-null   int64
18  touch_screen         2000 non-null   int64
19  wifi                 2000 non-null   int64
20  price_range          2000 non-null   int64
dtypes: float64(2), int64(19)
```

**Slika 2.** Osnovne informacije o datasetu

## 2.2 Uporabljeni modeli

### 2.2.1 Decision Tree

Kot prvi model sem uporabila Decision Tree. Gre za enostaven klasifikacijski algoritem, ki podatke razdeli v drevesno strukturo na podlagi vhodnih značilnosti. Model je enostaven za razlago in omogoča hitro klasifikacijo novih primerov.

Model sem naučila na učni množici, nato pa uporabila za napovedovanje razredov na testni množici. Uspešnost modela sem ocenila z natančnostjo (accuracy), matriko zamenjav (confusion matrix) in classification reportom.

```
#Decision tree model
from sklearn.tree import DecisionTreeClassifier

detr=DecisionTreeClassifier(random_state=42)
detr.fit(x_train, y_train)#treniram model (feature
s, correct class)

#napovedovanje na testni množici
napy=detr.predict(x_test)

print(napy[:7])#prikazujem prvih 7 napovedi

[3 2 0 2 3 2 0]

#ocenujem modela DT
from sklearn.metrics import accuracy_score
accdet=accuracy_score(y_test, napy)
print("DT accuracy:", accdet)

DT accuracy: 0.83
```

**Slika 3.** Učenje in osnovna ocena modela Decision Tree

```
In [17]: #Random Forest model
from sklearn.ensemble import RandomForestClassifier
randf=RandomForestClassifier(n_estimators=100, random_state=42)#berem si 100 dreves za uporabo ker je pogosta zacetna izbira
randf.fit(x_train, y_train)

Out[17]:
RandomForestClassifier(random_state=42)

In [18]: #napovedovanje z Random Forest modelom
naprf=randf.predict(x_test)
print(naprf[:7])

[3 1 0 2 3 2 0]

In [19]: #ocenujem model RF
accrandf=accuracy_score(y_test,naprf)
print("RF accuracy:", accrandf)
```

**Slika 4.** Učenje in osnovna ocena modela Random Forest

### 2.2.2 Random Forest

Po analizi z modelom Decision Tree sem uporabila še Random Forest. Ta model temelji na več odločitvenih drevesih, katerih napovedi se združijo v končno odločitev. Zaradi uporabe več dreves je model običajno bolj stabilen in manj občutljiv na prenaučanje kot posamezno odločitveno drevo.

Za model sem uporabila 100 dreves (`n_estimators = 100`) in parameter `random_state = 42`, ki omogoča ponovljivost

rezultatov. Po učenju modela sem napovedala cenovne razrede na testni množici in rezultate ocenila z natančnostjo, matriko zamenjav ter classification reportom.

Ena izmed prednosti modela Random Forest je možnost določanja pomembnosti posameznih značilk. Analizirala sem, katere tehnične lastnosti mobilnih telefonov imajo največji vpliv na napoved cenovnega razreda.

Rezultati so pokazali, da je najpomembnejša značilnost količina delovnega pomnilnika (RAM). Med pomembnejšimi značilnostmi so tudi zmogljivost baterije, širina in višina zaslona, masa telefona ter notranji pomnilnik.

Slika 5. Pomembnost značilk pri modelu Random Forest

```
In [32]: #dodatna analiza
#pomembnost značilk pri Random Forest modelu
featureimp=accplot.DataFrame({"Feature": x.columns, "Importance": randf.feature_importances_})
featureimp=featureimp.sort_values(by="Importance", ascending=False)
featureimp.head(7)
```

Out[32]:

|    | Feature       | Importance |
|----|---------------|------------|
| 13 | ram           | 0.480768   |
| 0  | battery_power | 0.072976   |
| 12 | px_width      | 0.056089   |
| 11 | px_height     | 0.056000   |
| 8  | mobile_wt     | 0.039007   |
| 6  | int_memory    | 0.034837   |
| 16 | talk_time     | 0.031891   |

### 2.2.3 MLPClassifier

Pred uporabo modela MLP sem vhodne podatke standardizirala s pomočjo razreda StandardScaler. Standardizacija je pomembna pri nevronskih mrežah, saj imajo vhodne značilnosti različna območja vrednosti. S standardizacijo imajo vse značilnosti primerljiv obseg, kar omogoča hitrejšo in stabilnejšo učenje modela.

Za klasifikacijo sem uporabila model MLPClassifier z enim skritim slojem, ki vsebuje 50 nevronov. Model sem učila največ 500 iteracij, pri čemer sem uporabila parameter `random_state = 42` za ponovljivost rezultatov.

```
#standardizacija podatkov za MLP model
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
xtrain=sc.fit_transform(x_train)
xtest=sc.transform(x_test)
```

```
#Nevronska mreža (MLP)
from sklearn.neural_network import MLPClassifier

mlp=MLPClassifier(hidden_layer_sizes=(50,),max_iter=500,random_state=42)
mlp.fit(xtrain, y_train)
```

```
MLPClassifier(hidden_layer_sizes=(50,), max_iter=500, random_state=42)
```

```
#napovedovanje z MLP modelom
```

```
napmlp=mlp.predict(xtestsc)
print(napmlp[:7])
```

```
[3 1 0 2 3 2 0]
```

Po učenju modela sem izvedla napoved na testni množici ter uspešnost ocenila z natančnostjo, matriko zamenjav in classification reportom.

Slika 6. Standardizacija podatkov in učenje modela MLP

## 3. Rezultati

Po učenju vseh treh modelov sem primerjala njihove rezultate na testni množici. Za ocenjevanje uspešnosti sem uporabila natančnost (accuracy), matriko zamenjav (confusion matrix) in classification report.

```
In [15]: #katere razrede model pogosto zamenjuje(Confusion matrix)
from sklearn.metrics import confusion_matrix
conf=confusion_matrix(y_test, napy)
print(conf)
```

```
[[92  8  0  0]
 [13 74 13  0]
 [ 0 13 80  7]
 [ 0  0 14 86]]
```

```
In [16]: #podrobna ocena modela DT
from sklearn.metrics import classification_report
cr=classification_report(y_test,napy)
print(cr)
```

```
precision    recall  f1-score   support

0               0.88        0.92        0.90         100
1               0.78        0.74        0.76         100
2               0.75        0.80        0.77         100
3               0.92        0.86        0.89         100

accuracy               0.83
macro avg              0.83
weighted avg           0.83
```

Slika 7. Ocena uspešnosti modela Decision Tree

### 3.1 Decision Tree

Decision Tree je dosegel natančnost 83 %. Model je pravilno razvrstil večino primerov, vendar je naredil največ napak pri srednjih cenovnih razredih.

### 3.2 Random Forest

Random Forest je dosegel boljše rezultate z natančnostjo 88 %. Zaradi uporabe več odločitvenih dreves je model dosegel boljšo stabilnost in manj napak kot Decision Tree.

```
[20]: #matrika zamenjav za Random Forest
      conmrf=confusion_matrix(y_test, naprf)
      print(conmrf)
```

```
[21]: #classification report (podrobna ocena modela RF)
      crmf=classification_report(y_test, naprf)
      print(crmf)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.95      | 0.96   | 0.96     | 100     |
| 1            | 0.82      | 0.84   | 0.83     | 100     |
| 2            | 0.81      | 0.79   | 0.80     | 100     |
| 3            | 0.93      | 0.93   | 0.93     | 100     |
| accuracy     |           |        | 0.88     | 400     |
| macro avg    | 0.88      | 0.88   | 0.88     | 400     |
| weighted avg | 0.88      | 0.88   | 0.88     | 400     |

**Slika 8. Ocena uspešnosti modela Random Forest**

### 3.3 MLP

Najboljše rezultate med osnovnimi modeli je dosegel MLP z natančnostjo 91,5 %. Model je pravilno klasificiral največ primerov in imel najmanj napačnih napovedi.

```
1: #matrika zamenjav MLP
   cmmlp=confusion_matrix(y_test, napmlp)
   print(cmmlp)
```

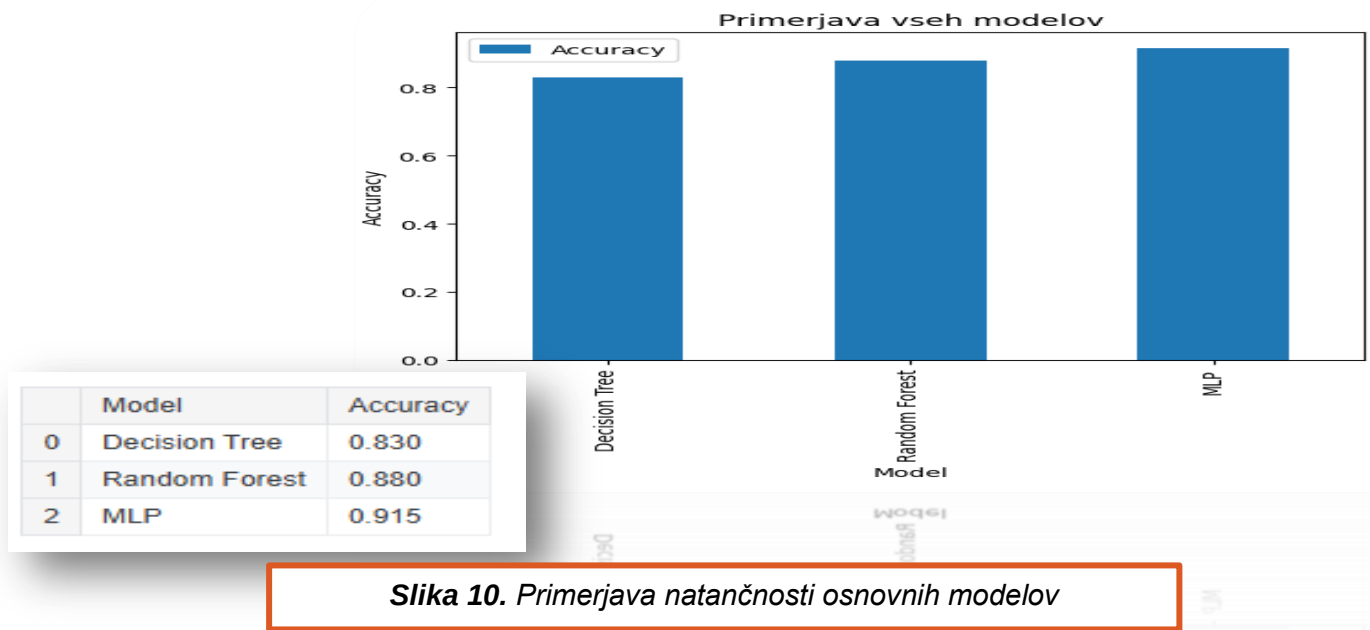
```
1: #podrobna ocena MLP modela
   crmlp=classification_report(y_test, napmlp)
   print(crmlp)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.95      | 0.93   | 0.94     | 100     |
| 1            | 0.91      | 0.90   | 0.90     | 100     |
| 2            | 0.87      | 0.92   | 0.89     | 100     |
| 3            | 0.94      | 0.91   | 0.92     | 100     |
| accuracy     |           |        | 0.92     | 400     |
| macro avg    | 0.92      | 0.92   | 0.92     | 400     |
| weighted avg | 0.92      | 0.92   | 0.92     | 400     |

**Slika 9. Ocena uspešnosti modela MLP**

### 3.4 Primerjava modelov

Primerjava rezultatov kaže, da se je z večanjem kompleksnosti modelov izboljšala tudi njihova uspešnost. MLP je dosegel najvišjo natančnost, Random Forest je bil drugi najboljši model, Decision Tree pa je dosegel najnižjo natančnost.



### 3.5 Dodatna analiza in izboljšava modelov

Po primerjavi osnovnih modelov sem dodatno analizirala možnost prenaučanja (overfitting). Zato sem primerjala natančnost modelov na učni in testni množici.

Pri vseh treh osnovnih modelih je bila natančnost na učni množici zelo visoka, kar kaže, da so modeli zelo dobro naučili učne podatke. Razlika med učno in testno natančnostjo pa nakazuje določeno stopnjo prenaučanja, predvsem pri modelu Decision Tree.

Za zmanjšanje prenaučanja sem preizkusila nekaj osnovnih izboljšav modelov.

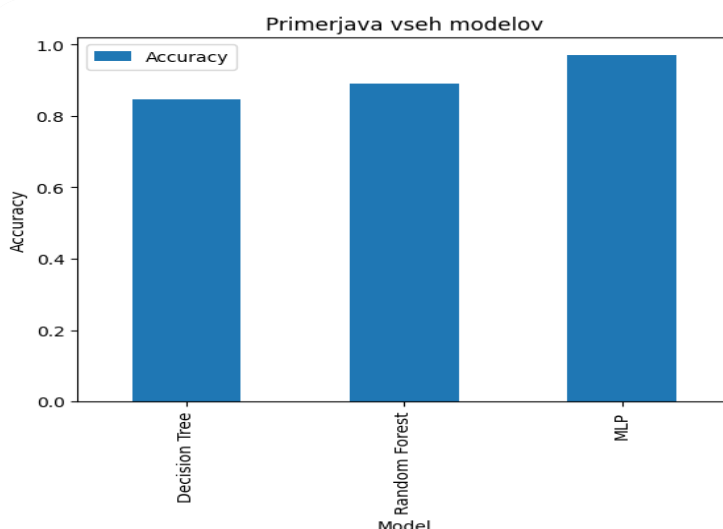
Pri modelu Decision Tree sem omejila globino drevesa z uporabo parametra `max_depth = 6`. Testna natančnost se je izboljšala z 83 % na 84,5 %.

Pri modelu Random Forest sem omejila globino posameznih dreves na `max_depth = 10`. Testna natančnost se je izboljšala z 88 % na 89 %.

Pri modelu MLP sem preizkusila različne vrednosti parametra `alpha`, ki predstavlja regularizacijo modela. Preizkusila sem vrednosti 0.01, 0.1, 0.5, 1, 1.5 in 2. Najboljši rezultat je dosegel model z `alpha = 2`, ki je dosegel 97 % natančnost na testni množici.

Rezultati kažejo, da lahko že manjše spremembe hiperparametrov izboljšajo sposobnost modelov za posploševanje na novih podatkih.

|   | Model         | Accuracy |
|---|---------------|----------|
| 0 | Decision Tree | 0.845    |
| 1 | Random Forest | 0.890    |
| 2 | MLP           | 0.970    |



**Slika 11.** Primerjava natančnosti izboljšanih modelov

### 3.6 Razlaga rezultatov

Rezultati projekta so pokazali, da izbira modela pomembno vpliva na uspešnost klasifikacije cenovnih razredov mobilnih telefonov.

Decision Tree je bil najpreprostejši model in je dosegel najnižjo natančnost. Prednost tega modela je njegova enostavna razlaga, vendar je pokazal tudi največ znakov prenaučanja. Z omejitvijo globine drevesa se je njegova uspešnost nekoliko izboljšala.

Random Forest je dosegel boljše rezultate kot Decision Tree. Združevanje več odločitvenih dreves je zmanjšalo vpliv posameznih napačnih odločitev in izboljšalo natančnost modela. Analiza pomembnosti značilk je pokazala, da ima največji vpliv na napoved količina RAM-a, sledijo pa zmogljivost baterije, ločljivost zaslona in notranji pomnilnik.

Najboljše rezultate je dosegel model MLP. Standardizacija podatkov je omogočila učinkovito učenje nevronske mreže, dodatna regularizacija z uporabo parametra alpha pa je izboljšala sposobnost posploševanja. Končni model je dosegel 97 % natančnost na testni množici.

Na podlagi rezultatov lahko zaključim, da so naprednejši modeli dosegli boljše rezultate kot osnovni modeli. Kljub temu tudi Decision Tree ostaja uporaben zaradi enostavne razlage delovanja, medtem ko Random Forest in MLP ponujata višjo natančnost.

#### **4. Zaključek**

V projektu sem uspešno razvila in primerjala tri modele strojnega učenja za klasifikacijo cen mobilnih telefonov. Uporabila sem Decision Tree, Random Forest in umetno nevronske mreže MLP.

Vse modele sem ocenila z uporabo natančnosti, matrike zamenjav in classification reporta. Poleg tega sem analizirala pomembnost značilk pri modelu Random Forest ter primerjala uspešnost modelov na učni in testni množici.

Dodatno sem preizkusila nekaj osnovnih izboljšav modelov. Omejitev globine dreves je izboljšala rezultate modelov Decision Tree in Random Forest, pri modelu MLP pa je uporaba regularizacije z  $\alpha = 2$  prinesla najboljši rezultat. Izboljšani MLP je dosegel 97 % natančnost na testni množici.

Projekt je pokazal, da lahko z ustrezno izbiro modela in osnovno optimizacijo hiperparametrov dosežemo zelo natančno klasifikacijo cenovnih razredov mobilnih telefonov.